DATABRICKS · LAKEBASE

Lakebase 101

Operational Postgres on the Lakehouse — a shared, governed memory layer for Databricks & UiPath agents.

Suncorp AI CoE | CDP | Infosec | 60 minutes

AGENDA · 60 MINUTES

What we will cover

- **Context** — why this session, and what problem Lakebase solves for claims
- **Lakebase 101** — what it is, the autoscaling model, and where it fits
- **Reference architecture** — data flow: Source → Delta → Lakebase
- **Security & connectivity** — the identity chain and the UC / Postgres RBAC boundary
- **Lakeflow cost controls** — sync schedule, timeseries key, cost tagging
- **Live demo** — Data API vs direct SQL, and role vs ownership

SECTION 01

01

Context — why this session

CONTEXT

Why Lakebase, why now

- Agentic workflows need **low-latency operational state**, not just analytics
- In claims, **Databricks agents and UiPath agents must share the same claim state**
- Lakebase gives **Postgres speed** with **Unity Catalog governance**
- Goal: align **AI CoE, CDP and Infosec** on one safe reference pattern
- Outcome: a reviewed blueprint you can extend toward production

SECTION 02

02

Lakebase 101

LAKEBASE 101

What is Lakebase

- Fully-managed **PostgreSQL (OLTP)** built into Databricks
- Serverless **Autoscaling: 0.5–32 CU**, scales to zero when idle
- **Git-style branching** for instant dev / test copies
- Unity Catalog integration over the **open Postgres protocol**
- **Millisecond** reads and writes for apps, agents and serving

LAKEBASE 101

The autoscaling model & where it fits

Resource model

- `Project › Branch › Endpoint`
- Read-write + read-only endpoints
- Scale-to-zero — pay for what you use
- Branch from a point in time

Design principle

- An **acceleration / serving** layer
- **Not** the system of record
- Delta stays the source of truth
- A low-latency mirror of governed data

LAKEBASE 101

Branching & budget controls

Branch like git

- Copy-on-write clones of prod in seconds
- Isolated dev, test & preview
- Carries schema, data, RPCs — a full clone
- Drop when done — no cleanup drift

Costs stay controlled

- No compute until an endpoint is attached
- Cap CU + scale-to-zero per endpoint
- Gate who can create via project ACLs
- Tag + budget policy for chargeback

Let teams self-serve branches without risking production or the bill — we'll see this open the live demo.

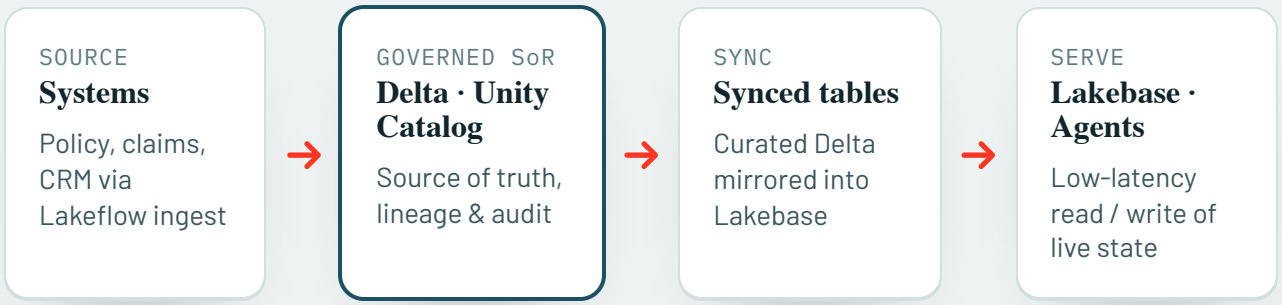
SECTION 03

03

Reference architecture — data flow

REFERENCE ARCHITECTURE

Source → Delta → Lakebase



Synced tables use a **primary key + timeseries key** to keep the latest row per key. Optional write-back flows agent state from Lakebase back to Delta for analytics.

REFERENCE ARCHITECTURE

Acceleration layer, not system of record

Lakebase IS

- A low-latency serving store
- Operational agent memory
- App state, features, sessions
- A governed mirror of Delta

Lakebase is NOT

- The source of truth
- A warehouse for BI / history
- Where you retain long-term data
- A bypass around UC governance

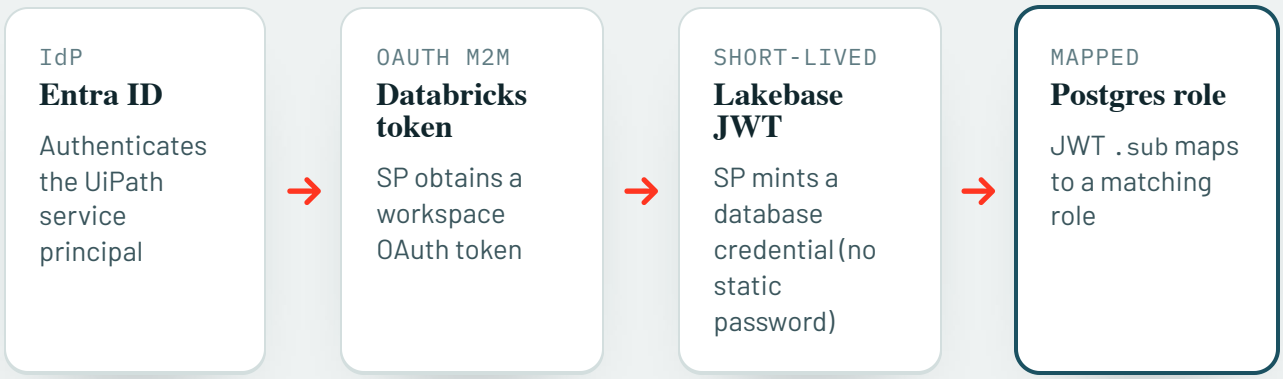
SECTION 04

04

Security & connectivity

SECURITY & CONNECTIVITY

Identity chain (documented — not shown live)



Follows the Suncorp identity design doc. Presented as reference architecture; the live demo uses the field-eng workspace.

SECURITY & CONNECTIVITY

UC / Postgres RBAC boundary

Unity Catalog plane

- Catalogs, schemas, lineage
- CDF history tables
- Databricks grants + audit
- Governs Delta & CDF outputs

Postgres RBAC plane

- Roles, RLS, `GRANT` / `REVOKE`
- Security-definer RPCs
- Data API role mapping
- Governs live Lakebase rows

Two planes, one identity — the same principal is governed differently on each side of the boundary.

SECURITY & CONNECTIVITY · DEMO CONCEPT

Role vs ownership

Authenticated principal

- The SP / user identity
- Mapped to a Postgres role
- Least privilege: `EXECUTE` only
- Recorded in every audit row

Object owner

- Owns the tables and RPCs
- `SECURITY DEFINER` bypasses RLS
- Runtime never owns tables
- Data API needs `databricks_create_role`

SECTION 05

05

Lakeflow cost controls

LAKEFLOW COST CONTROLS

Sync policy & cost attribution

Sync policy + cadence

- `SNAPSHOT` / `TRIGGERED` / `CONTINUOUS` modes
- Continuous = freshest, highest cost
- Triggered or scheduled for batch freshness
- Right-size cadence to the freshness need

Cost attribution

- Tag jobs, warehouses & Lakebase
- Chargeback by team / use case
- **Scale-to-zero** cuts idle spend
- Monitor via UC system tables

SECTION 06 · LIVE

06

Live demo — field-eng workspace

LIVE DEMO · START HERE

Branch production — clone it, safely

- Create a dev branch from production — **instant, copy-on-write**
- It carries the same schema, RPCs and agent memory — a full point-in-time clone
- Attach a **budget-safe** endpoint — small CU cap + scale-to-zero
- Experiment on dev; **production is untouched** — true isolation
- Throw the branch away when done — storage only until an endpoint runs

Verified live: dev cloned 5 tables + 4 RPCs + the seeded memory; a write on dev left production at 2 rows.

LIVE DEMO

Suncorp claims-center · agent memory

- Two agents share claim-scoped memory: `databricks.claim_triage.v1` & `uipath.document_review.v1`
- Runtime identities call **approved RPCs** — never touching tables directly
- Every operation writes an **append-only audit event** (reads included)
- **CDF** exposes row changes as Unity Catalog Delta history
- **Cross-claim access is denied** — boundaries fail closed

LIVE DEMO

Data API vs direct SQL

Data API (PostgREST)

- HTTPS + OAuth bearer token
- RPC + CRUD, **no Postgres driver**
- Ideal for UiPath / low-code
- Role via `databricks_create_role`

Direct SQL

- `psql` / `psycopg` + OAuth cred
- Full SQL and transactions
- Apps, notebooks, migrations
- Same roles, RLS and RPCs

LIVE DEMO · VERIFIED

What the demo proves

CHECK	PATH	RESULT
Read / write / task RPCs	`Data API · as service principal`	200 OK
Cross-claim completion	`Data API · as service principal`	403 · enforced
Direct table access	`Data API · as service principal`	403 · enforced
Principal vs logical agent	`Append-only audit event`	both recorded
Row-change history	`Unity Catalog CDF`	queryable

Green means the boundary behaved correctly — including the two `403`s, where denial is the intended, verified outcome.

WRAP-UP

Security takeaways

- Runtime principals get `EXECUTE` on approved RPCs only — **no table DML**
- RLS is enabled **fail-closed**; direct access returns nothing or is denied
- Security-definer RPCs enforce explicit **claim + business-unit** checks
- Audit records both the **authenticated principal** and the **logical agent id**
- CDF gives immutable **write history** in UC; reads are audited separately

WRAP-UP

Recommendations & next steps

- Use **separate service principals per agent runtime** for strong attribution
- Keep Lakebase as the **serving layer**; Delta remains the system of record
- Standardize the identity chain with **Infosec** (Entra → OAuth → PG role)
- Set sync cadence & cost tags with **CDP** for freshness and chargeback
- Review the SQL + RPC pattern with **security** before production

THANK YOU

Questions & discussion

Lakebase 101 · Suncorp AI CoE, CDP & Infosec — the claims-center agent-memory demo is a synthetic, non-production starter, reviewed end-to-end on the field-eng workspace.